

Computer Architecture

CS 622 / EE 520

Fall 2026

Lecture 6
Shahid Masud

- Pipelining in Computer Architecture
 - Basics of Pipelining
 - RISC-V Architecture Without Pipelining
 - Different Stages of a CPU Instruction Cycle
 - 5 stage pipeline
 - Pipeline Performance
 - Issues in Pipelined Architecture
 - Performance Penalty Reasons
 - Bubble Insertion
 - Instruction Reordering
 - Use of Forwarding
 - Architecture Modification needed
 - Problem with Branches
 - Introducing Branch Prediction

Slides Adapted from: (To read further)



Worcester Polytechnic Institute

Pipelining: Basic and Intermediate Concepts

Professor Hugh C. Lauer
CS-4515, System Programming Concepts

(Slides include copyright materials from Computer Architecture: A Quantitative Approach, 5th ed., by Hennessy and Patterson and from Computer Organization and Design, 4th ed. by Patterson and Hennessy)

CS-4515, D-Term 2015

Pipelining

1

Lecture 05 and 06: Pipeline: Basic/Intermediate Concepts and Implementation

CSCE 513 **Computer Architecture**

Department of Computer Science and
Engineering

Yonghong Yan

yanyh@cse.sc.edu

<https://passlab.github.io/CSCE513>

Lecture 07: RISC-V Single-Cycle Implementation

CSCE 513 **Computer Architecture**

Department of Computer Science and Engineering

Yonghong Yan

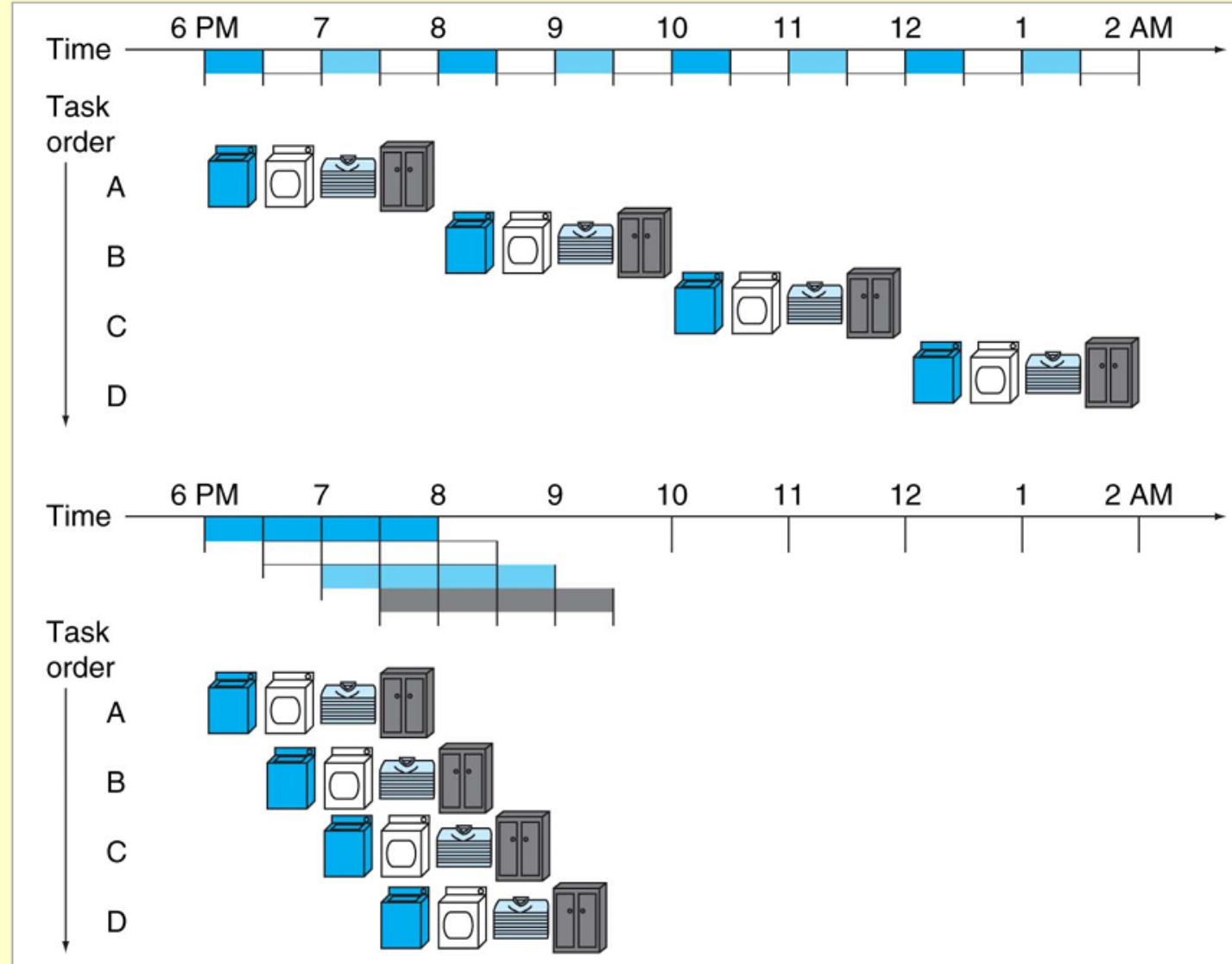
yanyh@cse.sc.edu

<https://passlab.github.io/CSCE513>

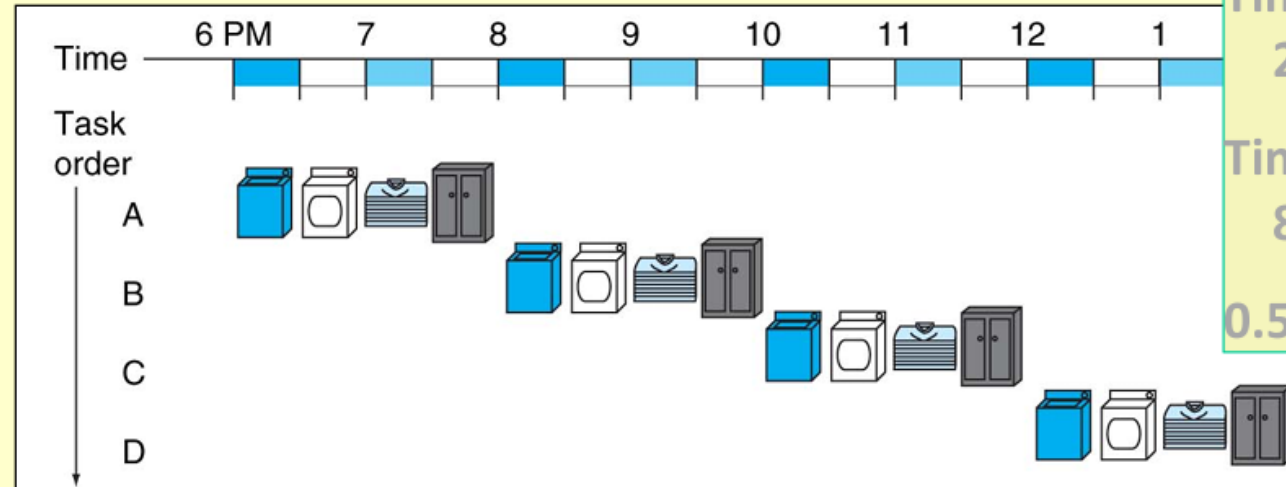
Readings:

Appendix **C** of course textbook

Clothes Wash Dry Iron Fold Example of Pipelining



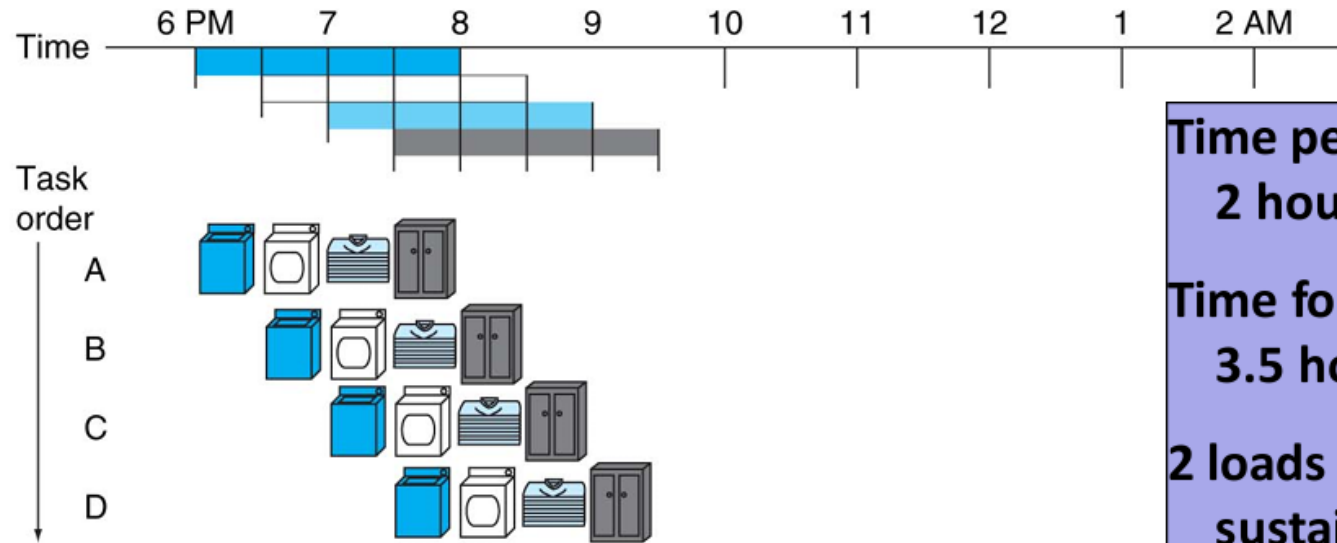
Processing Time Advantage



Time per load:—
2 hours

Time for 4 loads:—
8 hours!

0.5 Loads per hour

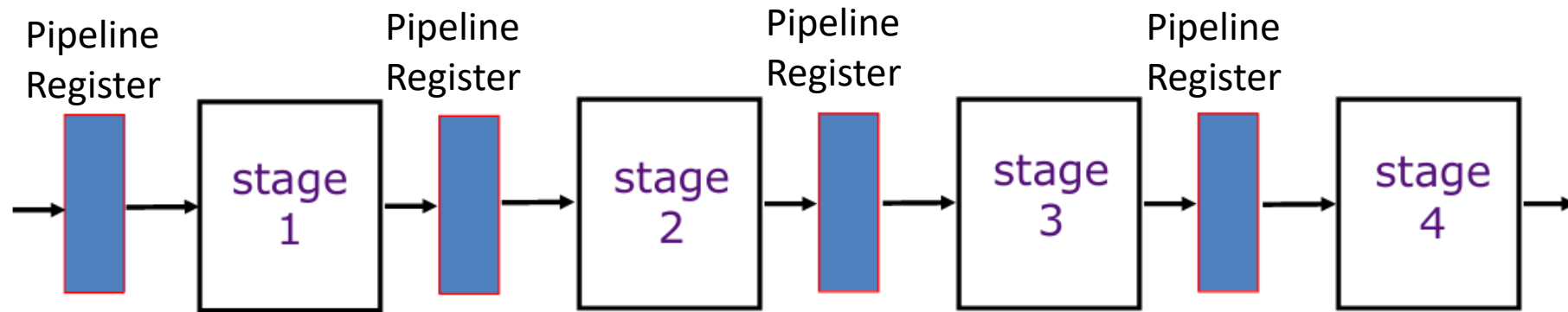


Time per load:—
2 hours (still)

Time for 4 loads:—
3.5 hours!

2 loads per hour
sustained rate!

An Ideal Pipeline has.....



- All objects go through the same stages
- No sharing of resources between any two stages
- Propagation delay through all pipeline stages is equal
- The scheduling of an object entering the pipeline is not affected by the objects in other stages

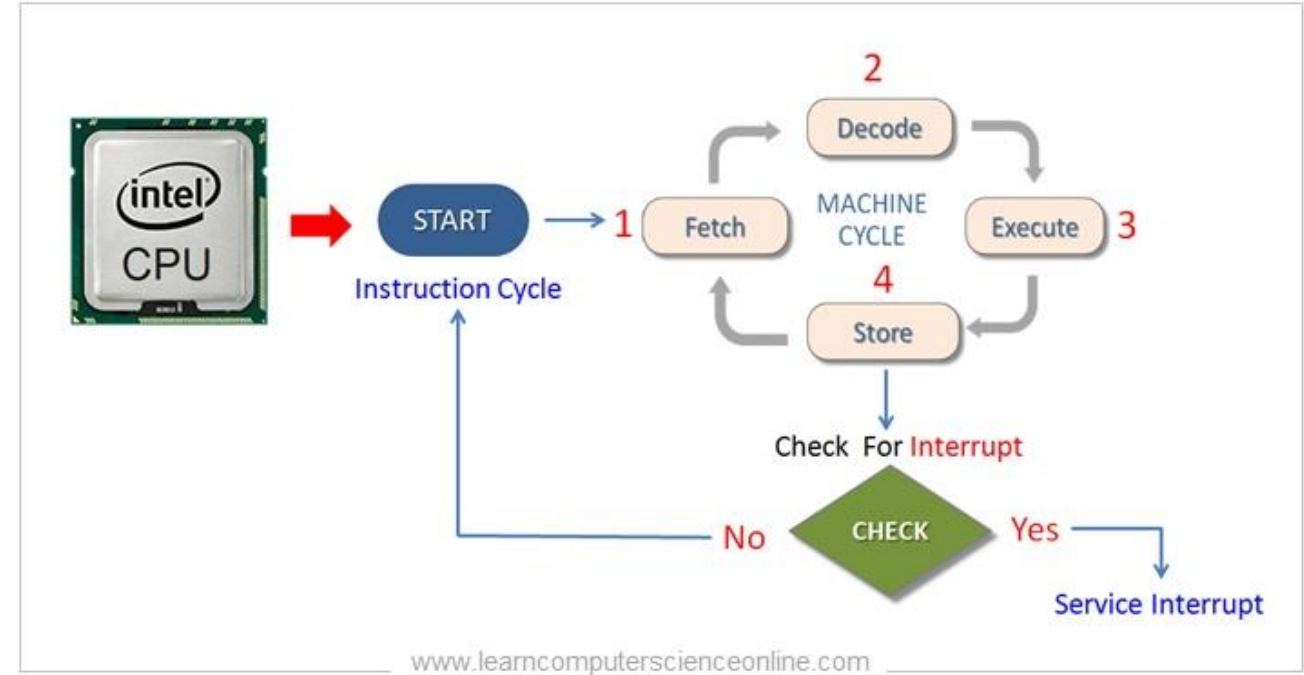
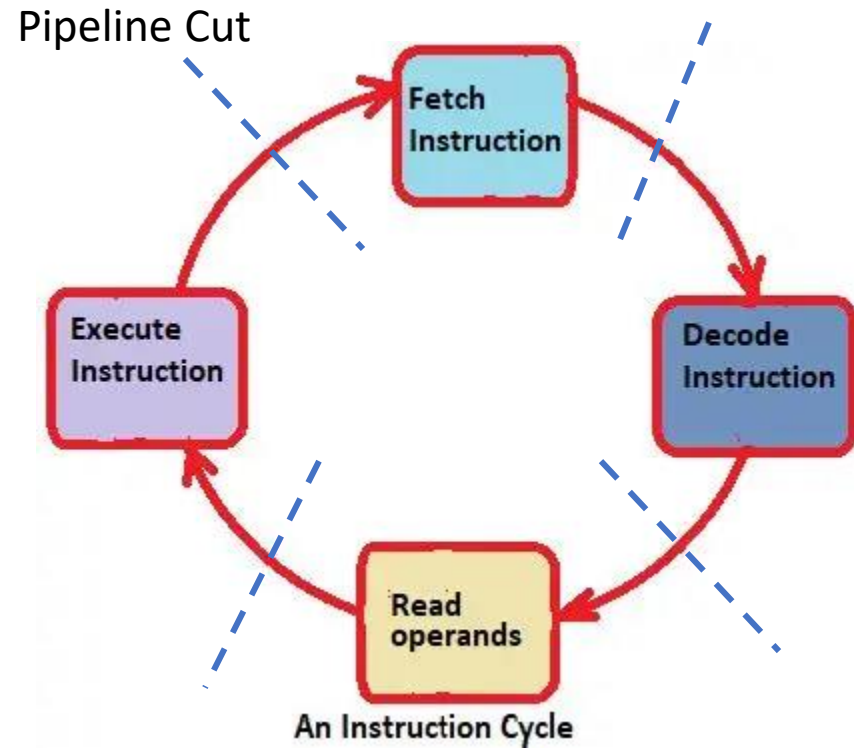
These conditions generally hold for industrial assembly lines

For laundry pipeline, two loads do not depend on each other.

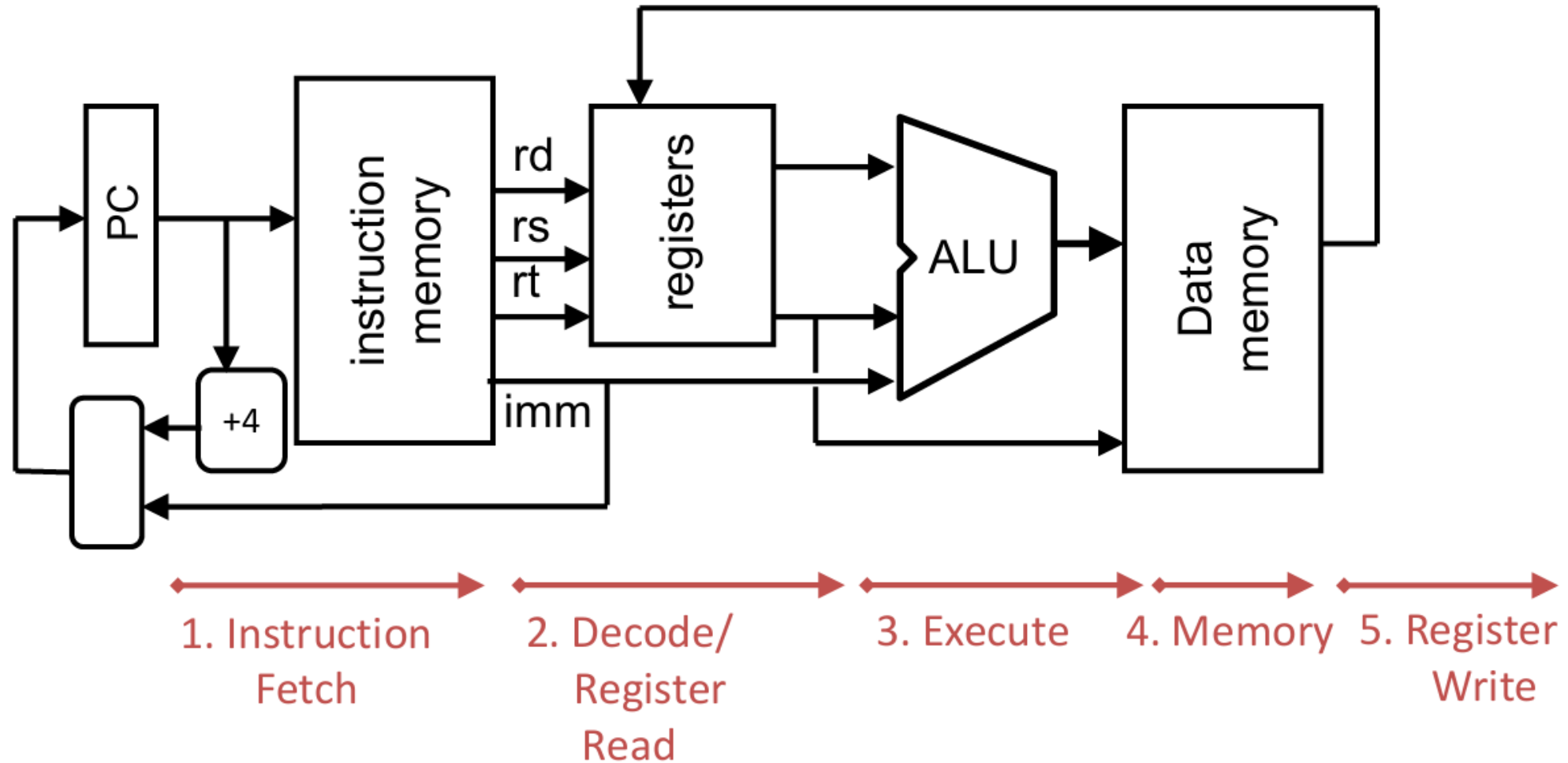
But instructions depend on each other!

Unpipelined CPU Architecture

A Computer Instruction Cycle



Stages of Instruction Execution in a CPU





Introducing Pipelining in CPU Architecture

- ❑ Partition Instruction Execution into distinct stages**
- ❑ Each Stage uses a different set of hardware resources**
- ❑ Stages can execute concurrently, in a pipelined style**

- **Three models of parallelism in computing...**

- Data parallelism
- Task parallelism
- Pipelining

- **... plus one new one**

- Google – massive parallelism (warehouse scale)

Example — MIPS Pipeline

■ Instruction fetch

- Increment instruction counter as part of this stage

■ Instruction decode and Register fetch

- I.e., read operand(s) from register(s)

■ Execution *or* Address Calculation

- Arithmetic operation using fetched registers or immediate operand

■ Memory access

- Read from or write to (main) memory

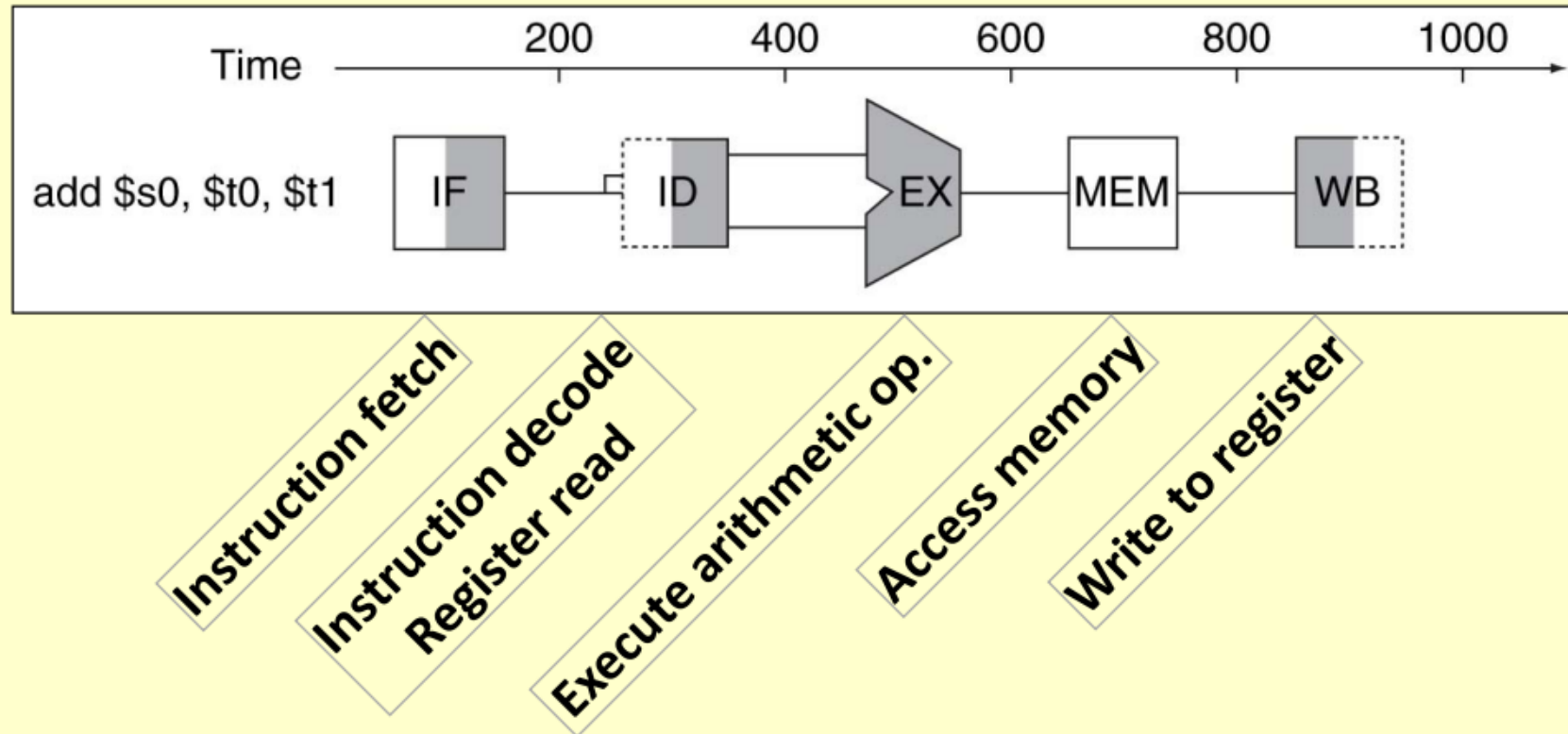
■ Write-back to Register

- Store result into register again

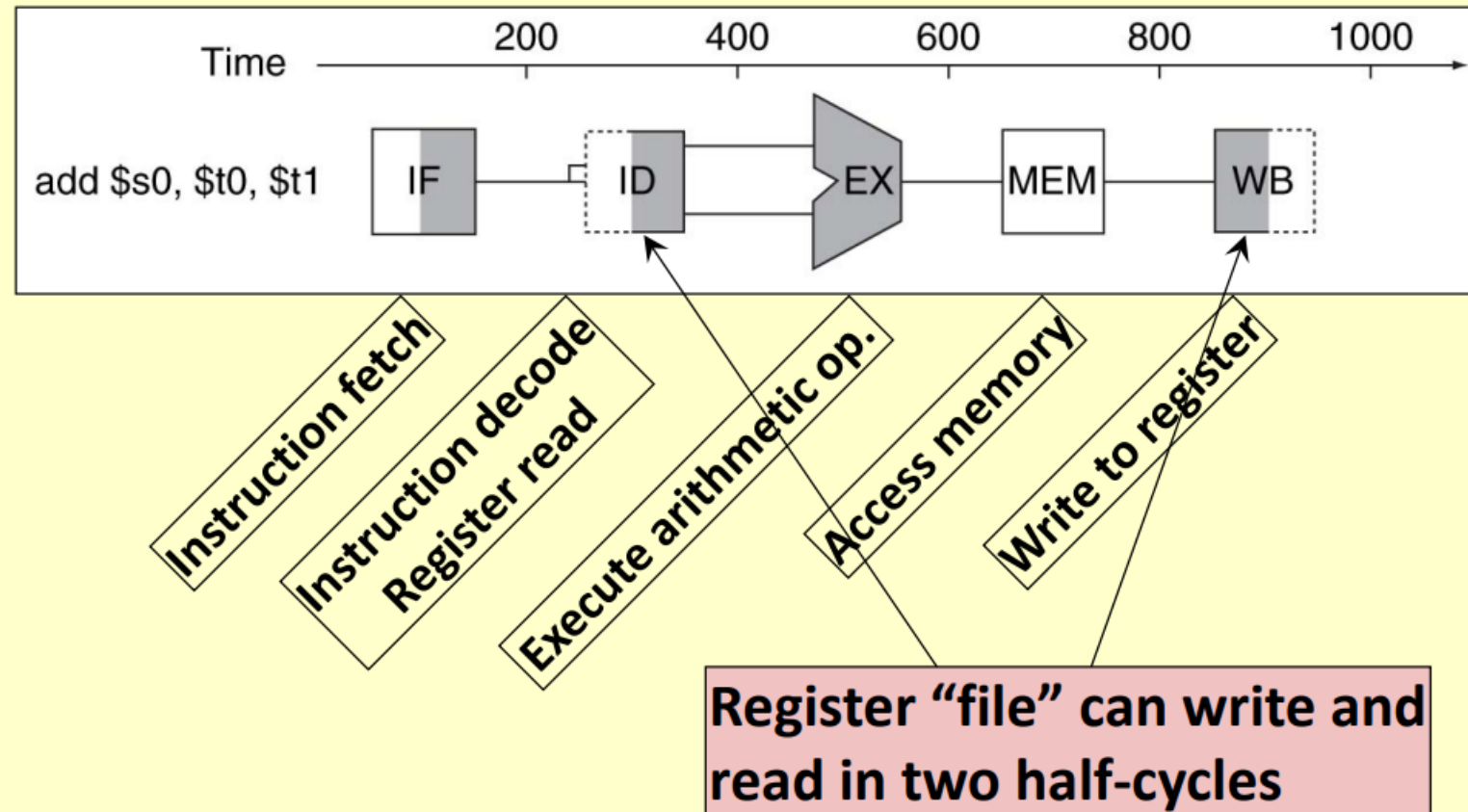
Every architecture partitions its instruction set differently

These are the five cycles of a many “textbook” pipelines!

Example — MIPS Pipeline (continued)

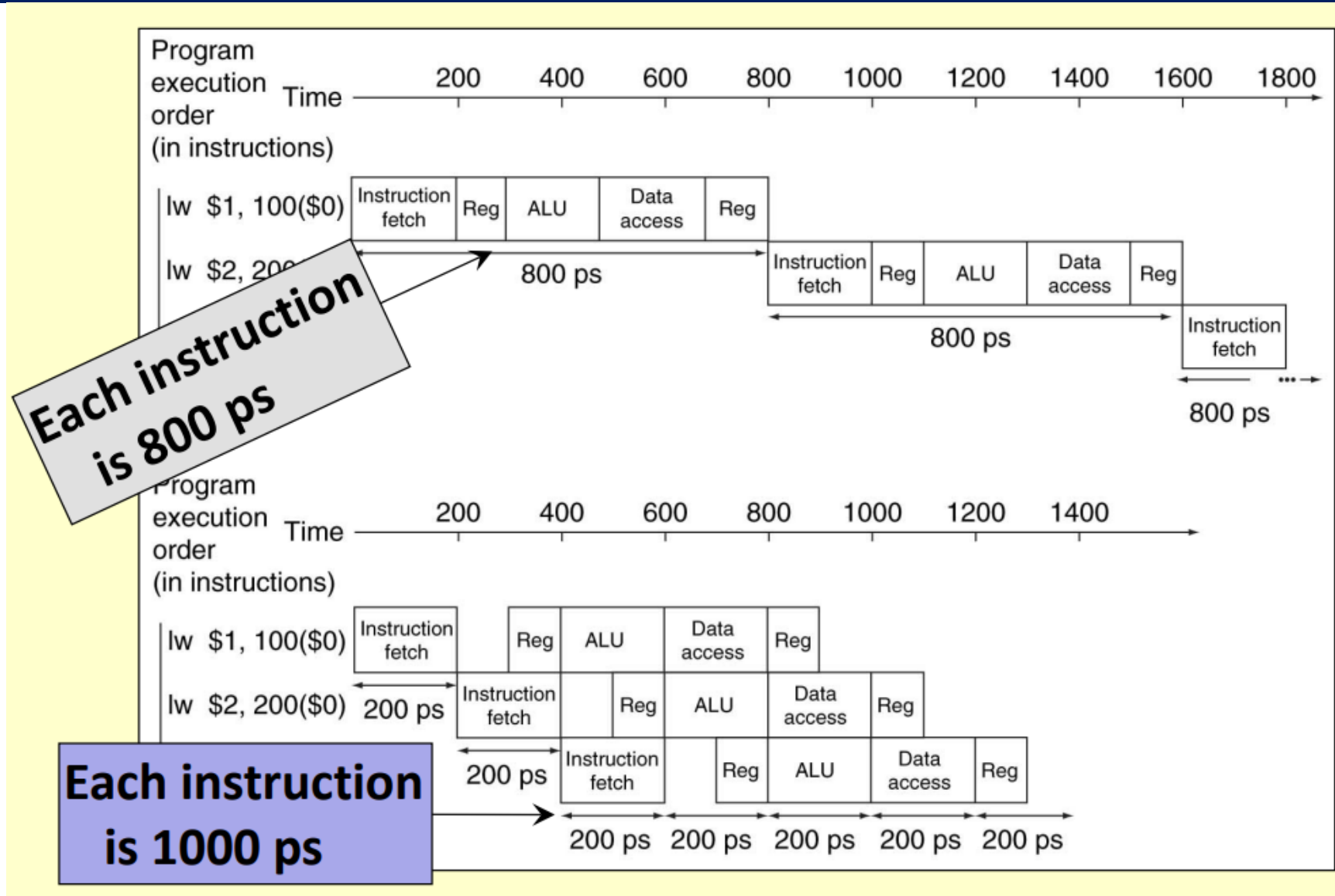


Example — MIPS Pipeline (continued)





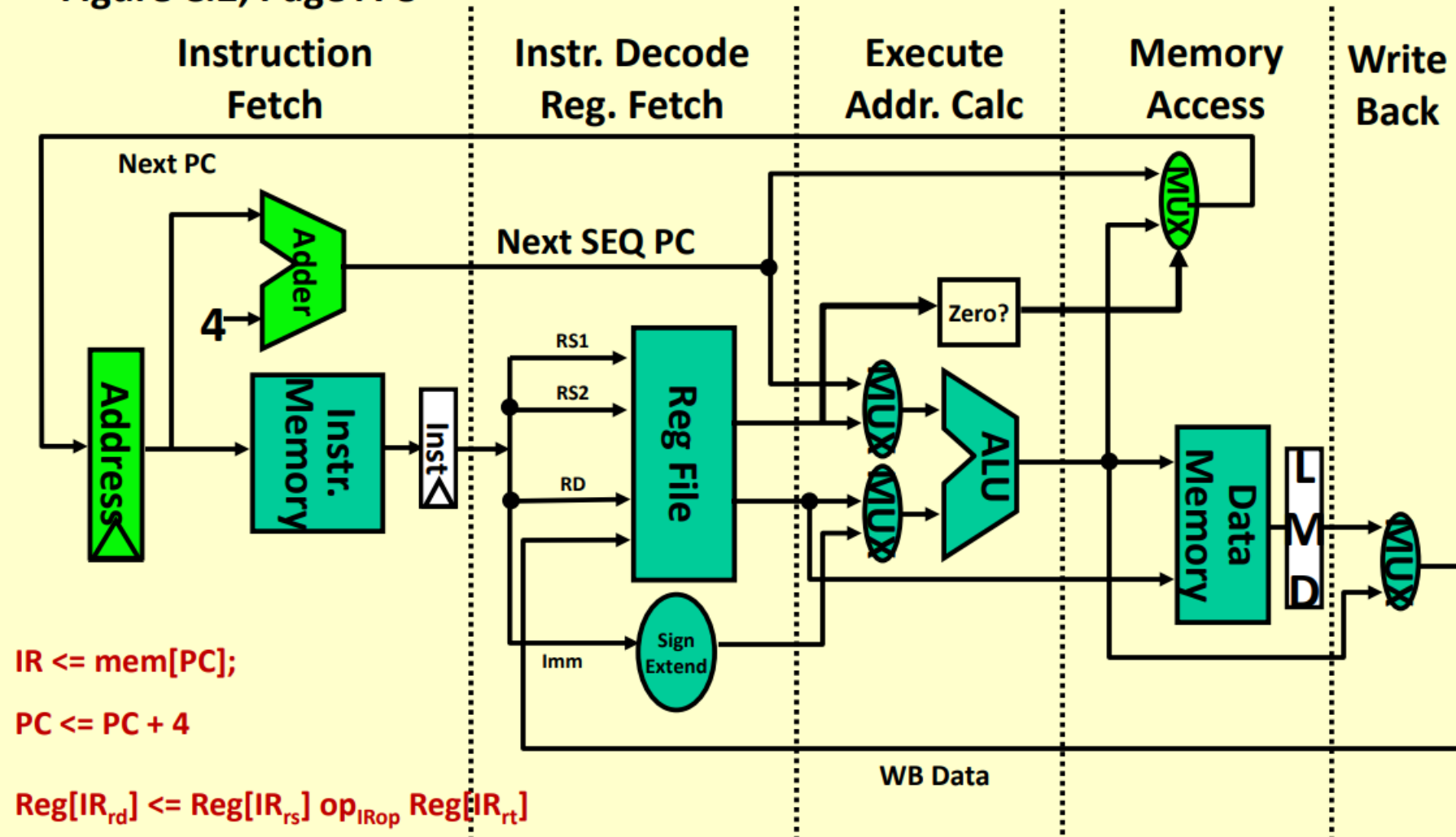
Pipeline Performance



What is a Datapath?

MIPS Datapath (in five steps)

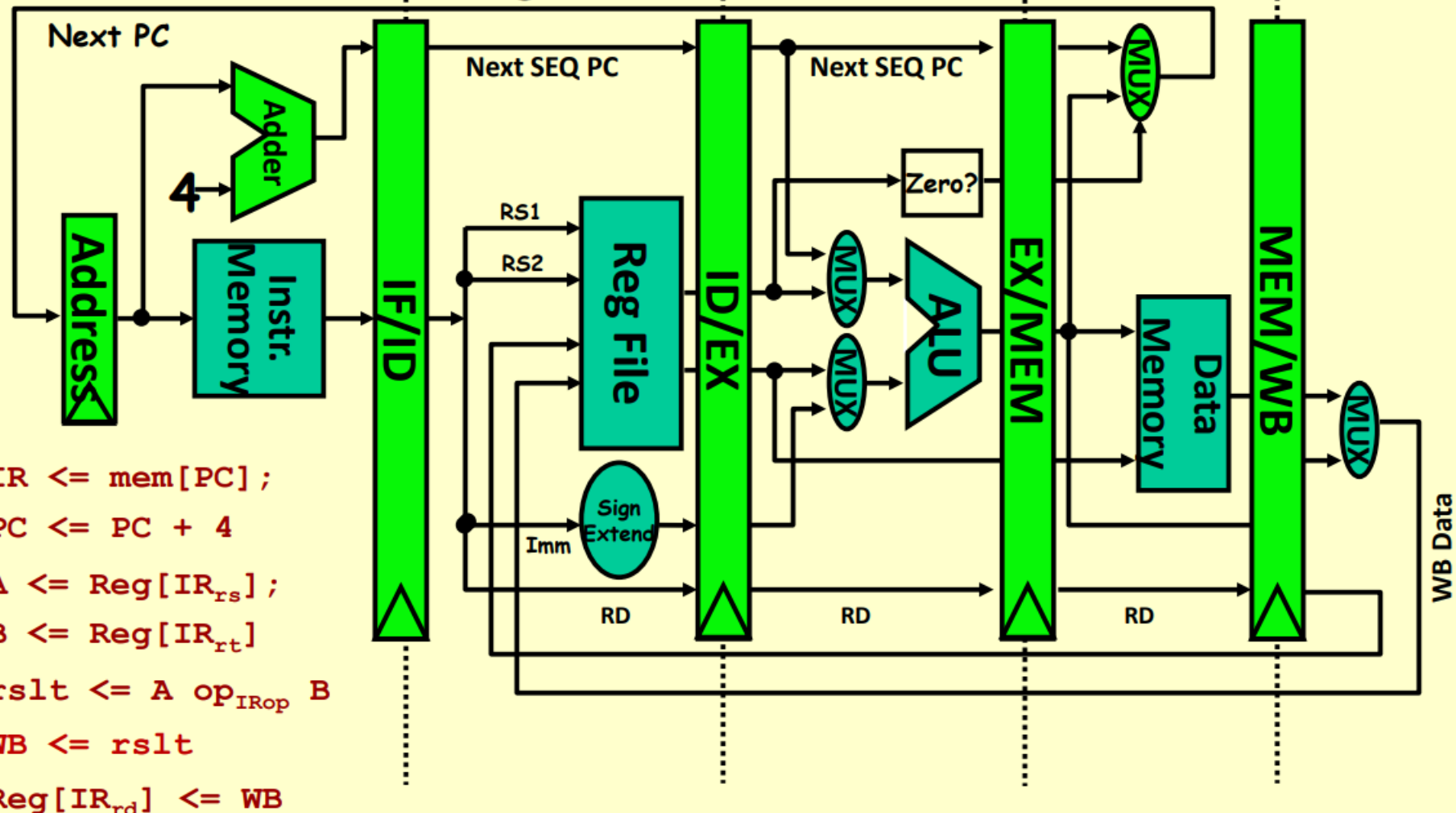
Figure C.2, Page A-8



Datapath with Pipeline Registers

MIPS Datapath (showing pipeline registers)

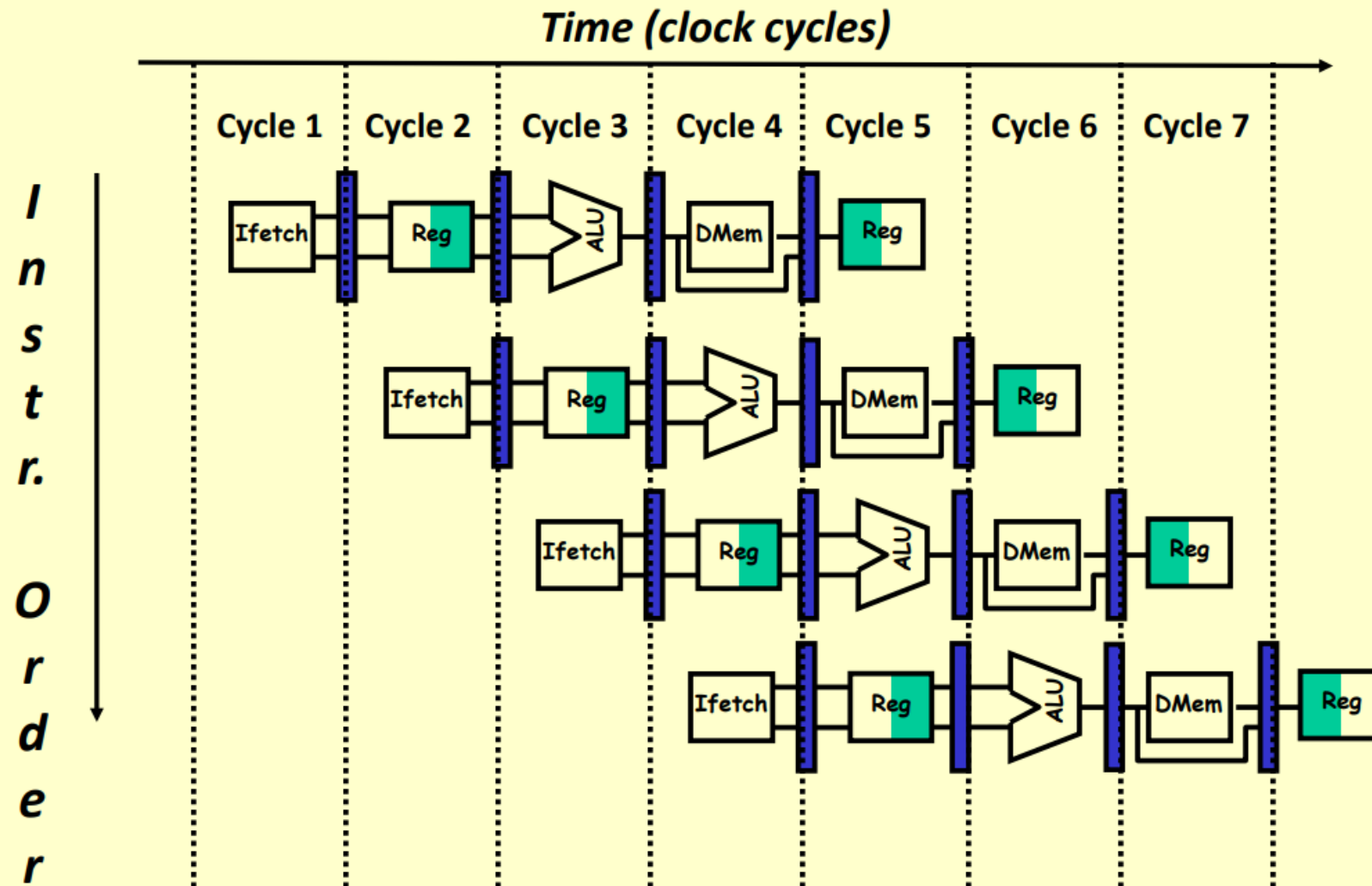
Figure C.3, Page C-9
Instruction
Fetch



Visualizing Pipelining through diagram

Visualizing Pipelining

Figure c.2, Page c-8



Quantifying the Speedup

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

■ Assume

- 40% ALU operations
- 20% branches
- 30% Load operations
- 10% Store operations
- No pipeline penalty

■ Average (non-pipelined) instruction duration = 650 ps

$$.4 \times 600 + .2 \times 500 + .3 \times 800 + .1 \times 700 = 650$$

Quantifying the Speedup (continued)

$$\text{Speedup} = \frac{\text{AveUnPipelinedTime}}{\text{AvePipelinedTime}} = \frac{650}{200} = 3.25$$

- Unpipelined architecture would allow variable duration instructions
 - Branches much faster than Loads
- Pipelined architecture requires every cycle to take exactly the same time
 - Some wasted time in Branch, ALU, and Store operations
- Speedup is less if there is a penalty for pipelining

More on Speedup

- Suppose that there is a penalty for including pipelining (vs. non-pipelined design)
- The clock cycle of each stage in pipelined architecture would be slightly longer
- Speedup would be less
- Subject to Amdahl's Law
- See pp. C-10 & C.11

Pipeline Hazards

Instructions Interact With Each Other in Pipeline: Dealing with Hazards

- An instruction may need a resource being used by another instruction → *structural hazard*
 - Solution #1: Stalling newer instruction till older instruction finishes
 - Solution #2: Adding more hardware to design
 - E.g., separate memory into I-memory and D-memory
 - Our 5-stage pipeline has no structural hazards by design
- An instruction depends on something produced by an earlier one
 - Dependence may be for a data value or for using same register (not the value) → *data hazard*
 - Solutions for RAW hazards: #1, interlocking (bubble delay), and #2, forwarding
 - WAR and WRW hazards: not possible for 5-stage pipeline
 - Dependence may be for the next instruction's address → *control hazard (branches, exceptions)*
 - Solutions: # delay, prediction, etc

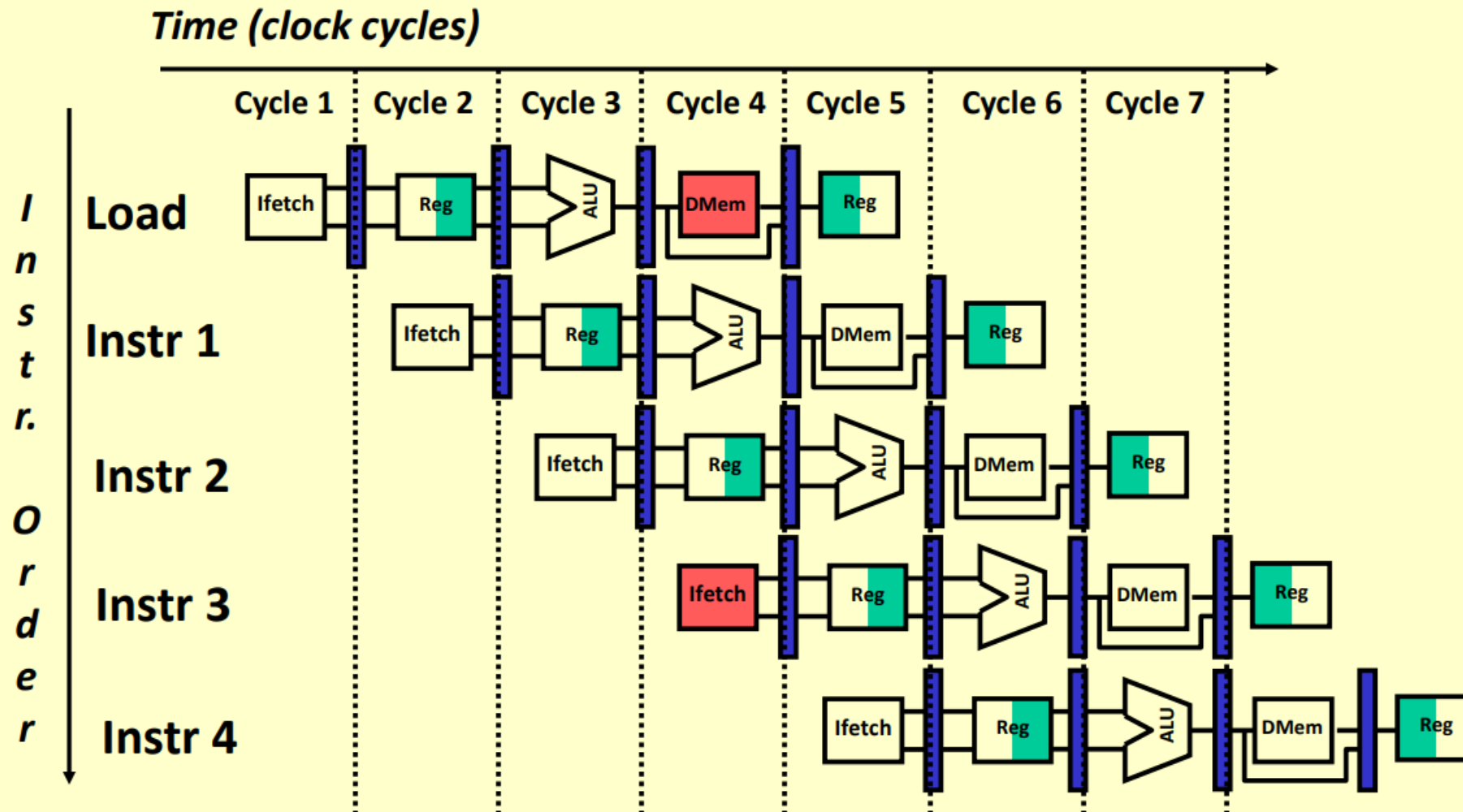
Hazards

- ***Structural*** – resource conflicts among successive instructions
 - Hardware cannot support all possible combinations of instructions in rapid succession
- ***Data*** – dependencies of instructions on results of other instructions
 - $x = a * b + c$ Cannot add until multiply is done
- ***Control*** – branches that change instruction fetch order
 - Will affect instructions that have already been fetched!

Structural Hazard due to Memory Access

Structural Hazard Example:–

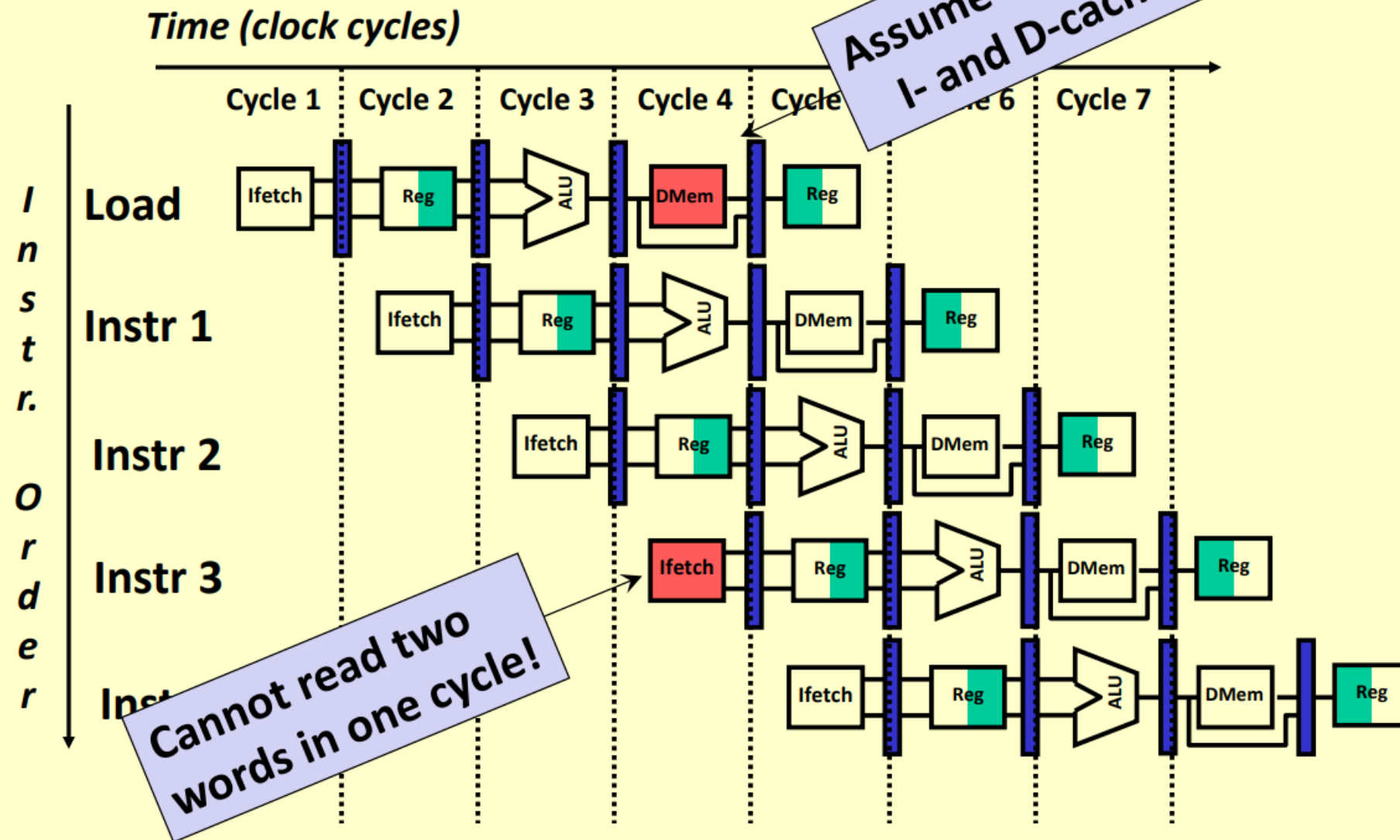
One Memory Port Figure C.4, Page C-14



Hazard due to One Memory Port

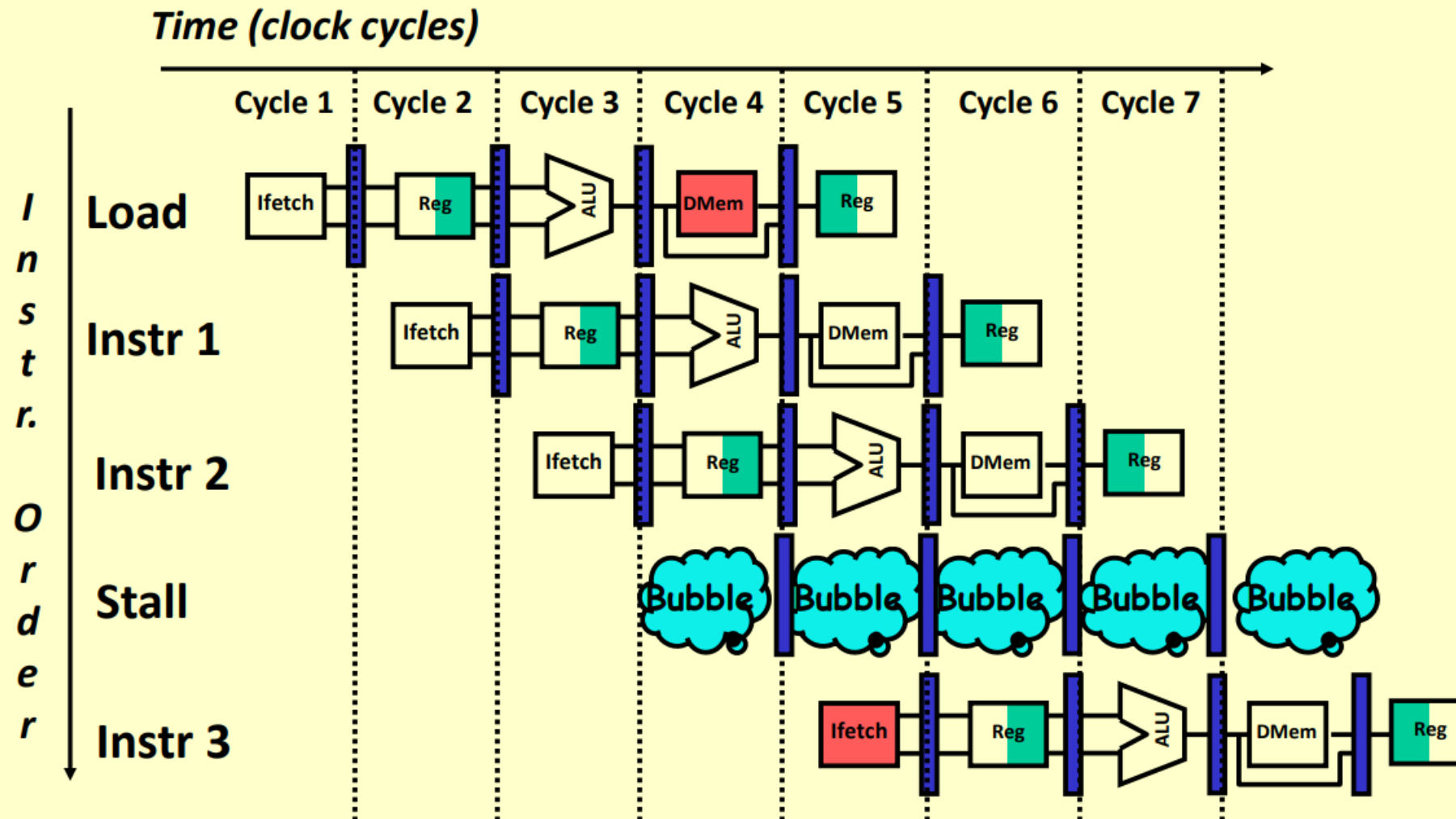
Structural Hazard Example:–

One Memory Port Figure C.4, Page C-14



Penalty – a Bubble in Pipeline

(Similar to Figure C.5, Page C-15)



Speedup Equation for Pipelining

See pp. C-12 & C-13

$$\text{CPI}_{\text{pipelined}} = \text{Ideal CPI} + \text{Average Stall cycles per Inst}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{Ideal CPI} + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

For simple RISC pipeline, CPI = 1

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

What to do about Memory Structural Hazard

- **Typical processor has two memory ports ...**
 - I-port
 - D-port

- **... or two separate L1 cache memories**
 - I-cache
 - D-cache

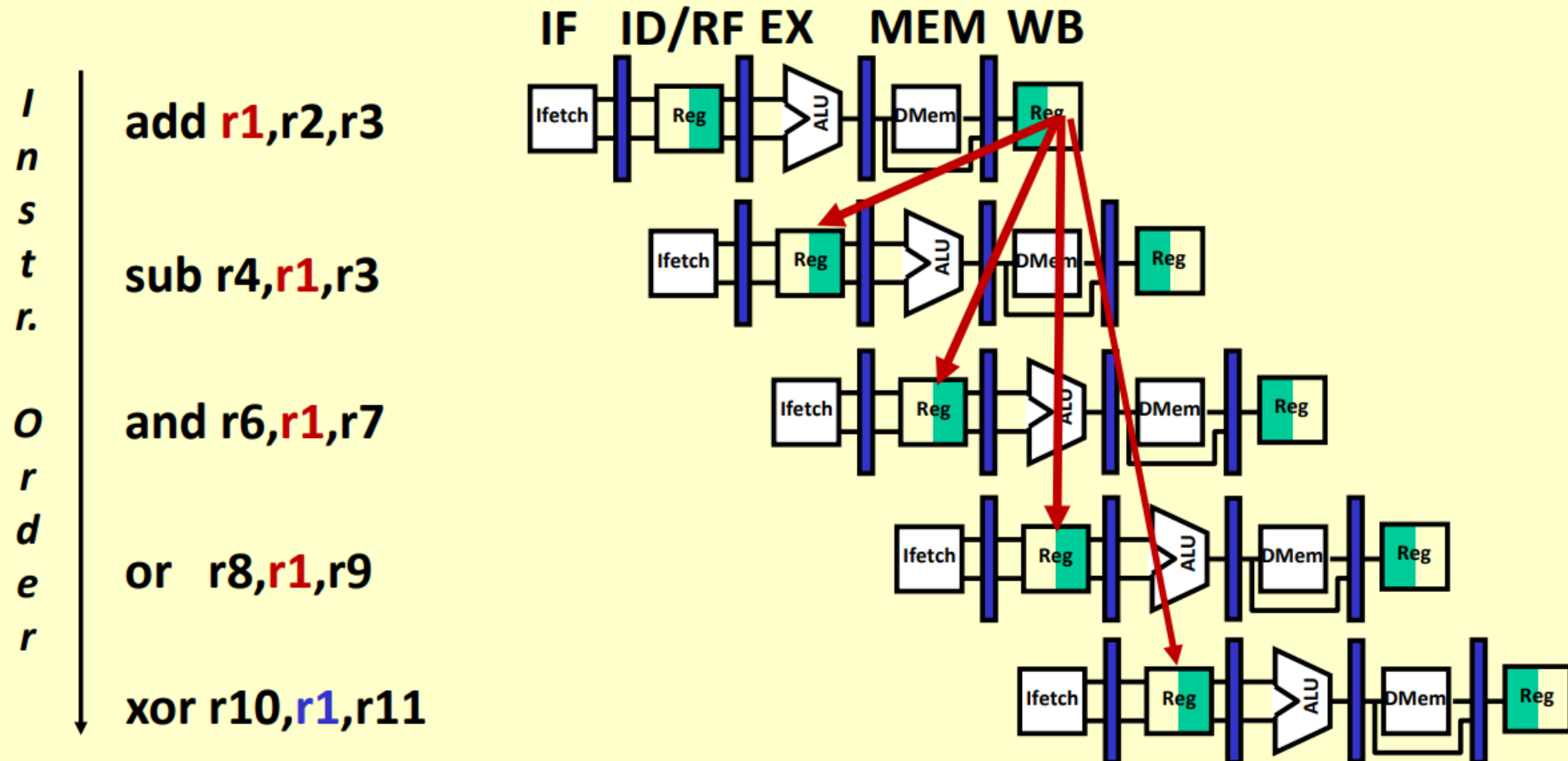
**Add Multiple Hardware Execution Blocks in Parallel
To Avoid Structural Hazard at EX Stage of Pipeline**

Data Related Pipeline Hazard

Data Hazard on R1

Figure C.6, Page C-17

Time (clock cycles)



3 Types of Data Hazard



■ Read After Write (RAW)

Instr_j tries to read operand before *Instr_i* writes it

 I: add **r1**, r2, r3
 J: sub r4, **r1**, r3

■ Write After Read (WAR)

Instr_j writes new value before *Instr_i* reads old value

 I: sub r4, **r1**, r3
 J: add **r1**, r2, r3
 K: mul r6, r1, r7

■ Write After Write (WAW)

Instr_j writes a value before *Instr_i* writes a different value

 I: sub **r1**, r4, r3
 J: add **r1**, r2, r3
 K: mul r6, r1, r7

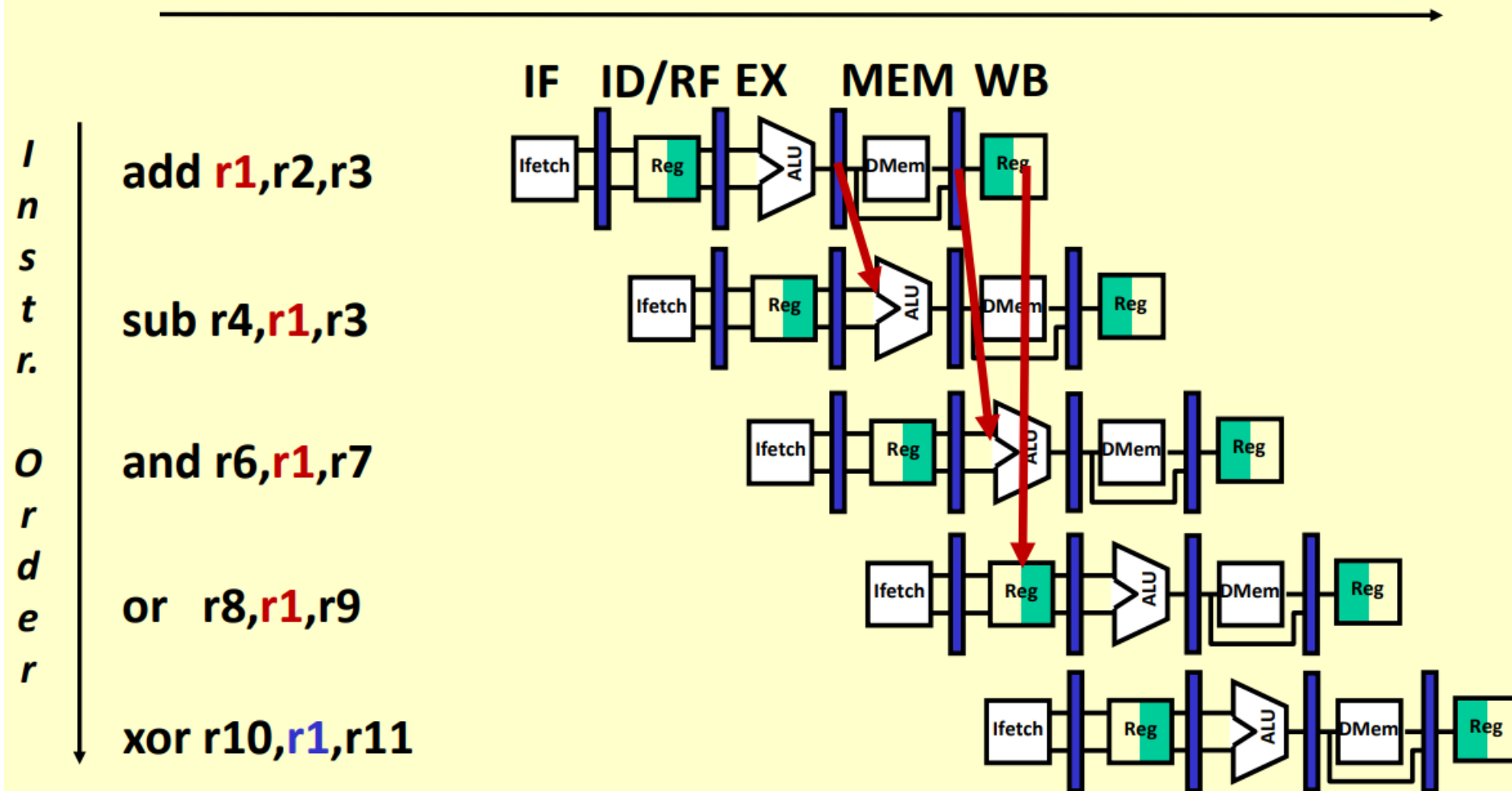
Forwarding in Pipeline Architecture

How does Forwarding Help?

Forwarding to Avoid Data Hazard

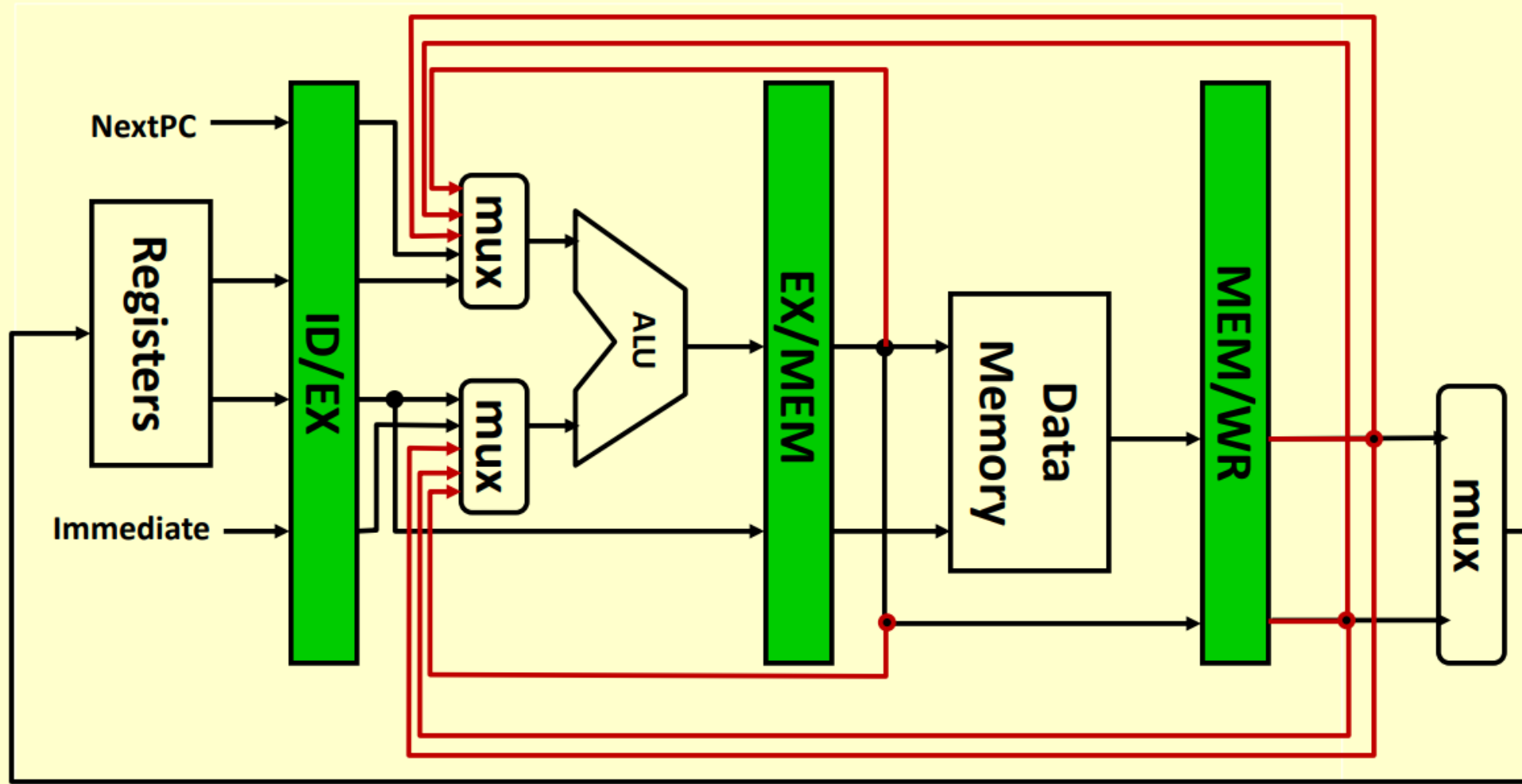
(Figure C.7, Page C-18)

Time (clock cycles)

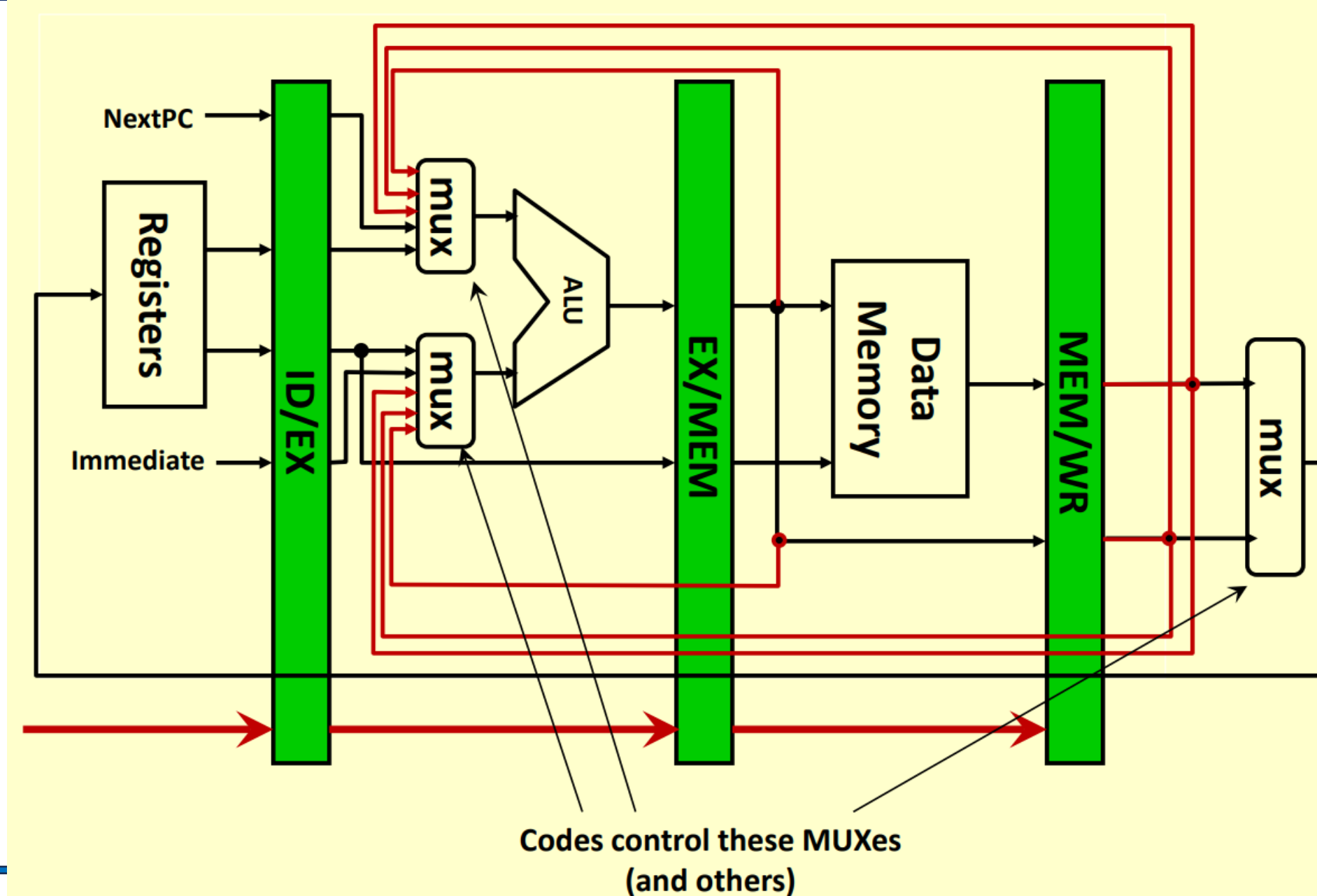


HW Change for Forwarding

Figure C.21, Page C-34



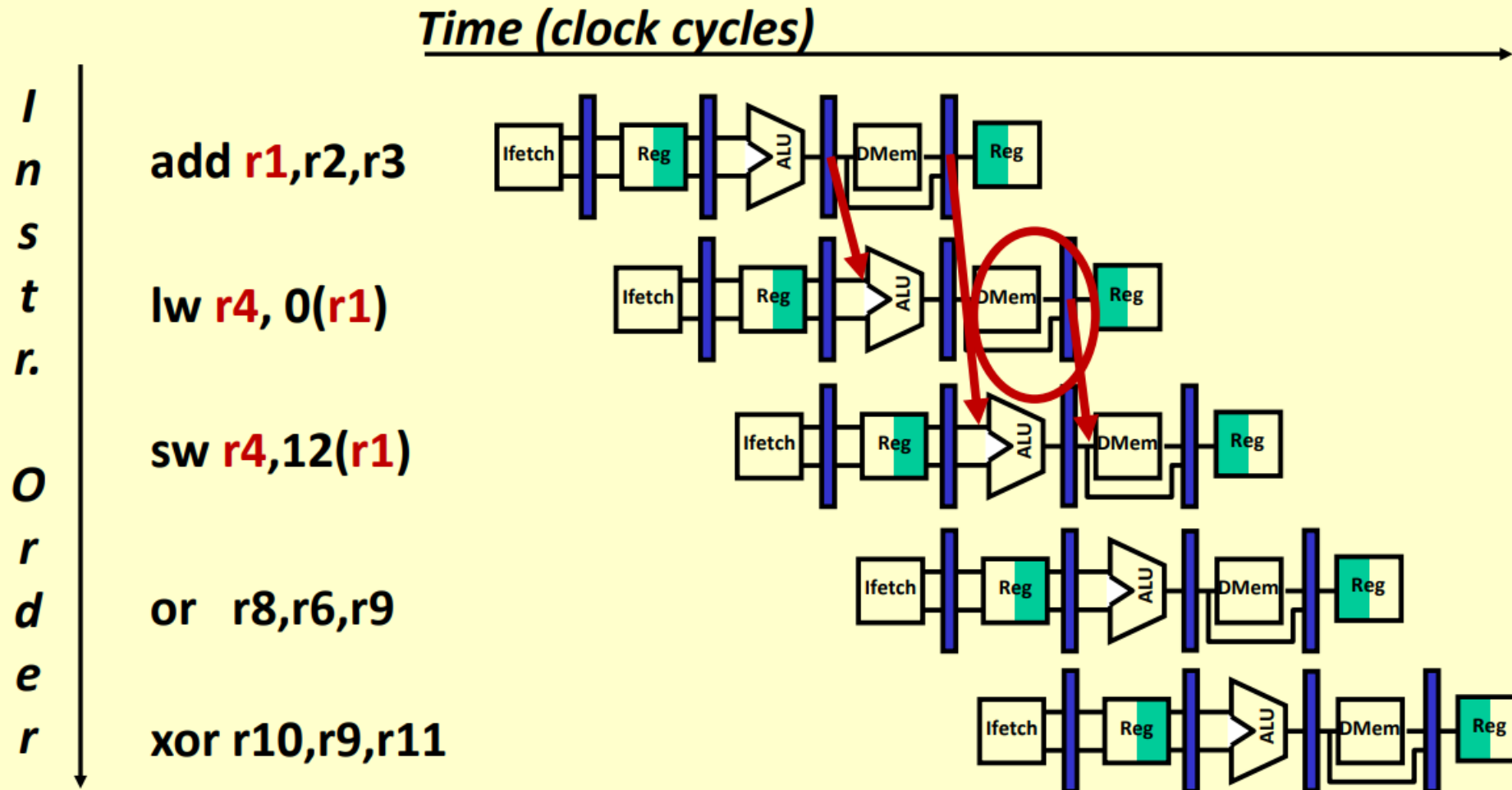
Corresponding Control Signals – Controlpath



Avoid Data Hazard in Forwarding

Forwarding to Avoid LW-SW Data Hazard

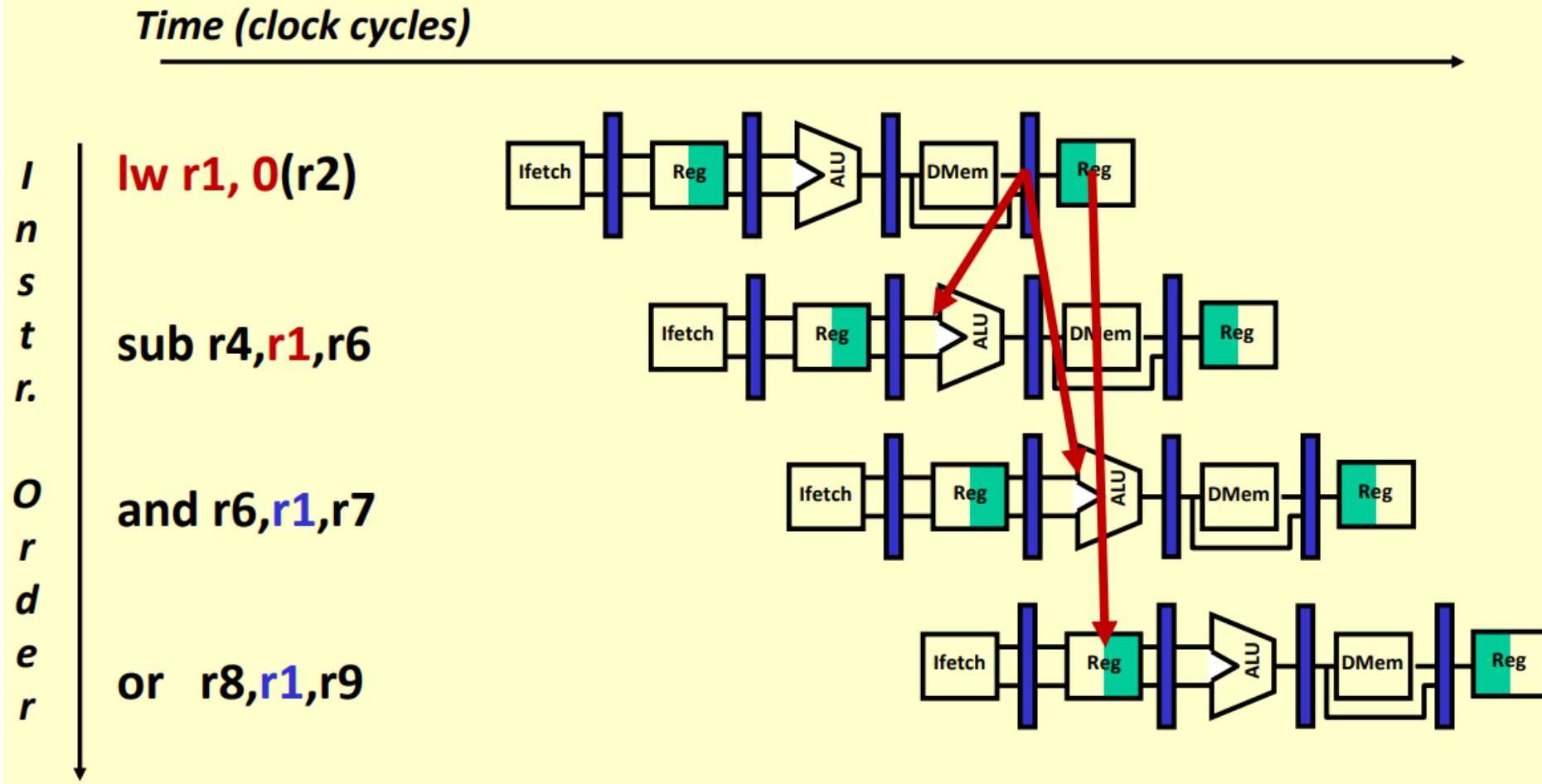
Figure C.8, Page C-19



Hazard Possible with Forwarding

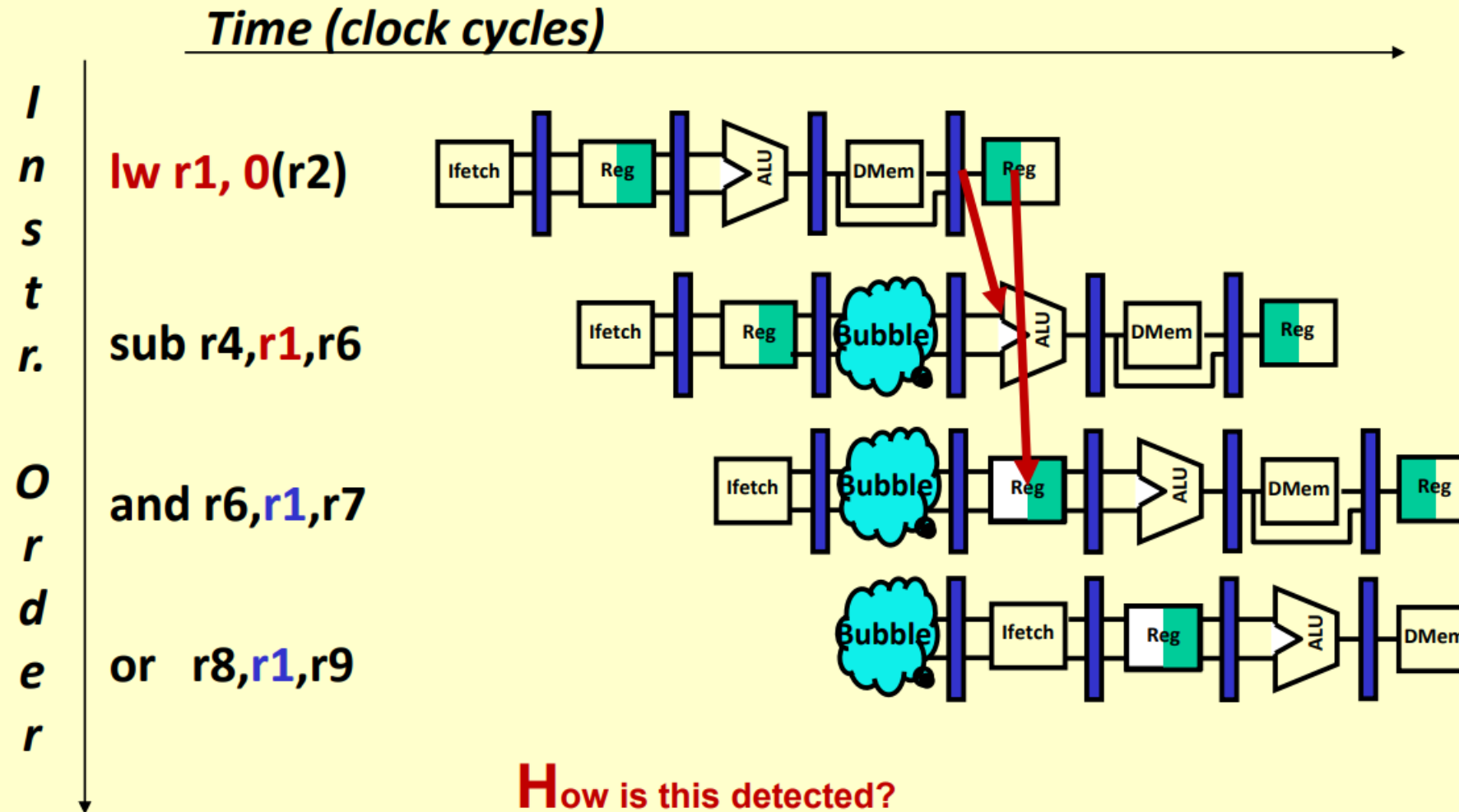
Data Hazard Even with Forwarding

(Figure C.9, Page C-20)



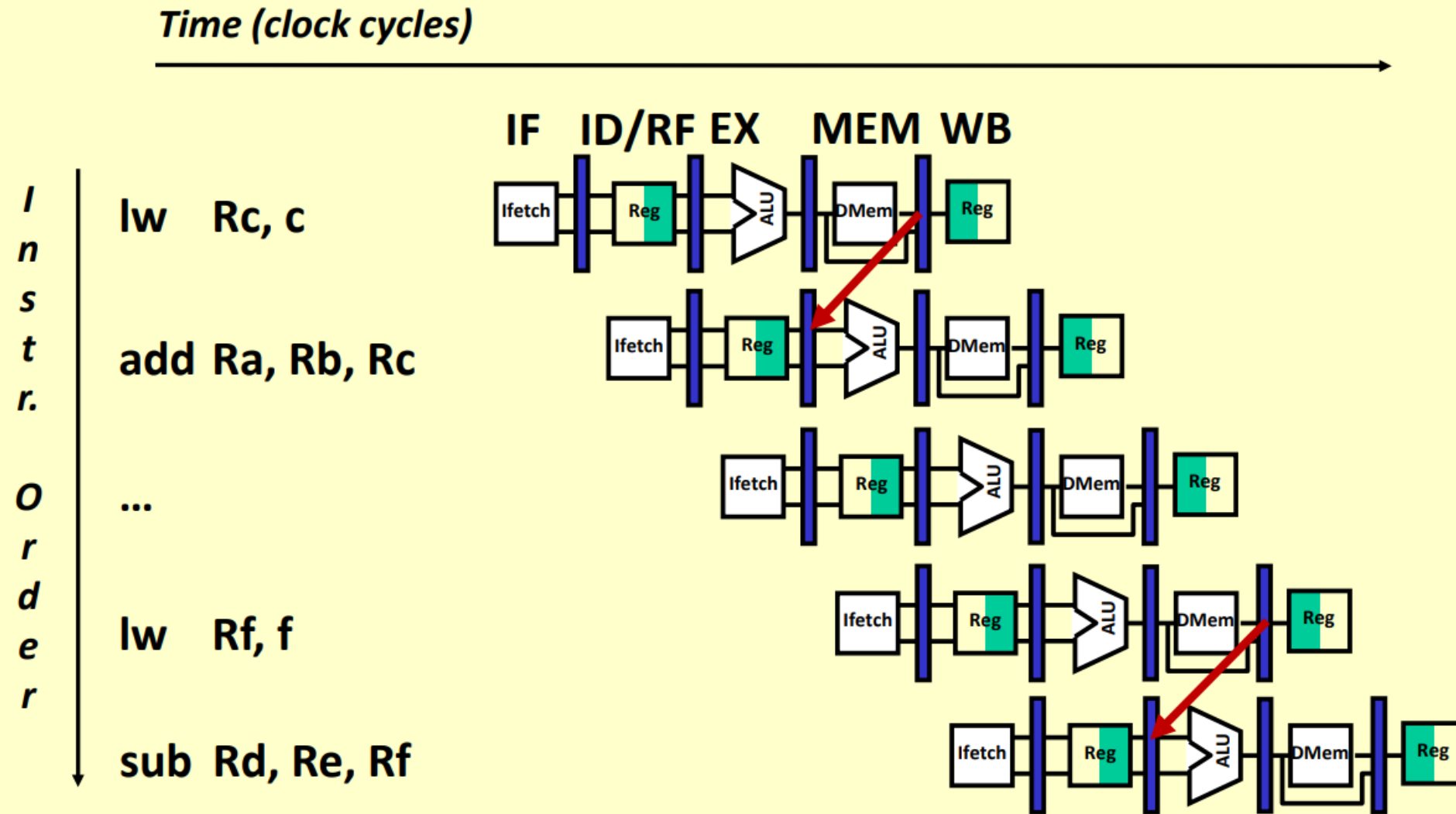
Data Hazard Causing Pipeline Stall

(See Figure C.10, Page C-21)



More on Data Hazard

Data Hazard (continued)



Instruction Reordering for Hazard Removal

Software Scheduling to Avoid Data Hazards

Try producing fast code for

$a = b + c;$

$d = e - f;$

assuming $a, b, c, d, e,$ and f in memory.

Slow code:

```
LW  Rb,b
LW  Rc,c
ADD Ra,Rb,Rc
SW  a,Ra
LW  Re,e
LW  Rf,f
SUB Rd,Re,Rf
SW  d,Rd
```

Fast code:

```
LW  Rb,b
LW  Rc,c
LW  Re,e
ADD Ra,Rb,Rc
LW  Rf,f
SW  a,Ra
SUB Rd,Re,Rf
SW  d,Rd
```

A useful instruction

A useful instruction

Compiler optimizes for performance.
Hardware checks and interlocks for safety.

How to decide

- **Keep track of three previous instructions**
- **If operand is *not* written by any of three previous instructions ...**
 - ... then read from register
- **If operand is output of ALU operation of *previous* instruction ...**
 - ... then forward from ALU stage
- **If operand is output from ALU or MEM of 2nd previous instruction ...**
 - ... then forward from MEM stage
- **Else stall one cycle**

This entire decision tree can be programmed into gates in hardware

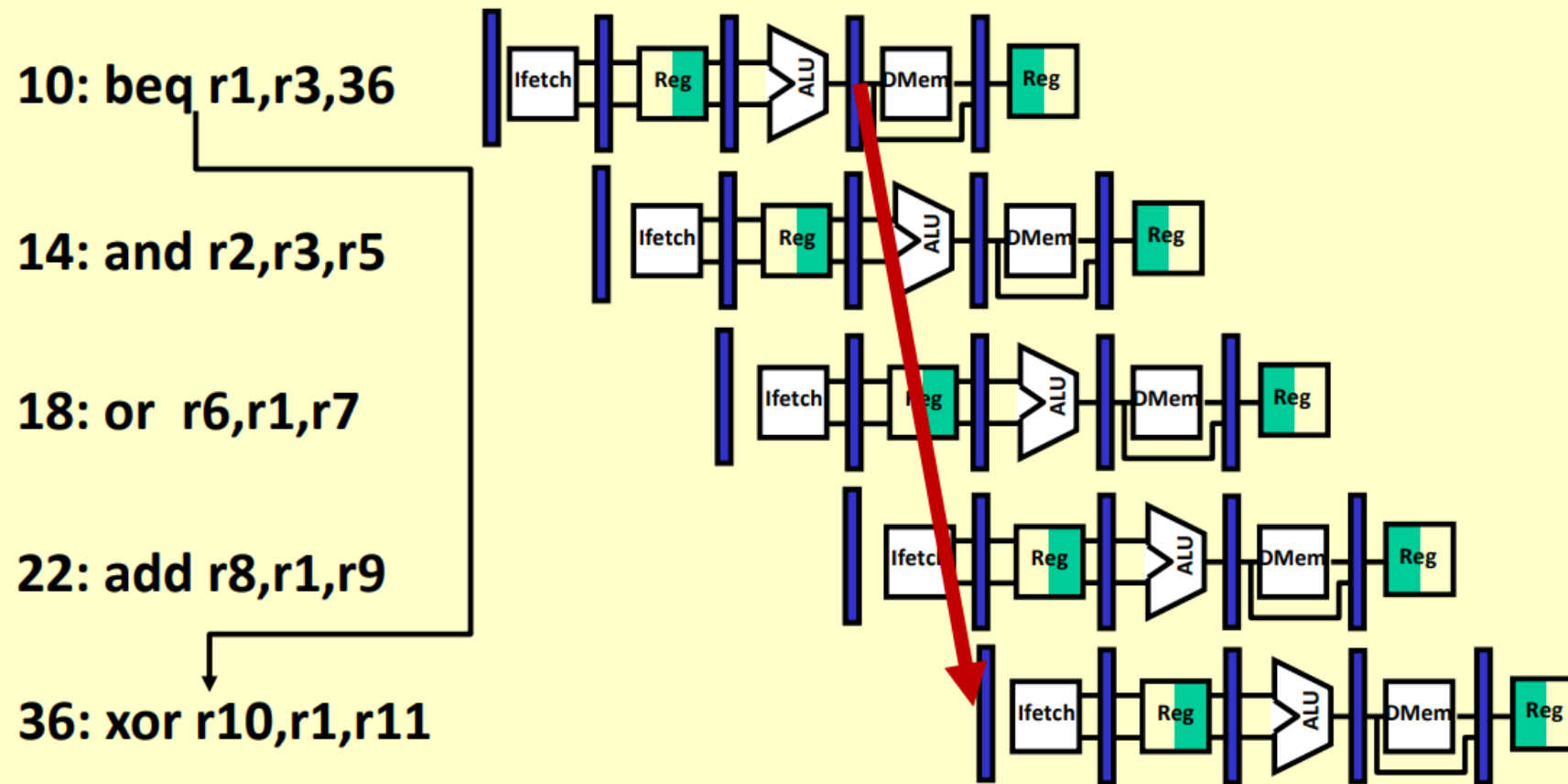
Control Hazards

Control Hazards

- Decision to take a *branch* or not may *not be known* until after the next instructions are fetched!

Control Hazard on Branches

Control Hazard on Branches (Three Stage Stall)



What do you do with the 3 instructions in between?

How do you do it?

Where is the "commit"?

Example of Pipeline Performance

- Example** Consider the unpipelined processor in the previous section. Assume that it has a 0.5 ns clock cycle) and that it uses four cycles for ALU operations and branches and five cycles for memory operations. Assume that the relative frequencies of these operations are 40%, 20%, and 40%, respectively. Suppose that due to clock skew and setup, pipelining the processor adds 0.1 ns of overhead to the clock. Ignoring any latency impact, how much speedup in the instruction execution rate will we gain from a pipeline?

Answer The average instruction execution time on the unpipelined processor is

$$\begin{aligned}
 \text{Average instruction execution time} &= \text{Clock cycle} \times \text{Average CPI} \\
 &= 0.5 \text{ ns} \times [(40\% + 20\%) \times 4 + 40\% \times 5] \\
 &= 0.5 \text{ ns} \times 4.4 \\
 &= 2.2 \text{ ns}
 \end{aligned}$$

In the pipelined implementation, the clock must run at the speed of the slowest stage plus overhead, which will be $0.5 + 0.1$ or 0.6 ns; this is the average instruction execution time. Thus, the speedup from pipelining is

$$\begin{aligned}
 \text{Speedup from pipelining} &= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}} \\
 &= \frac{2.2 \text{ ns}}{0.6 \text{ ns}} = 3.7 \text{ times}
 \end{aligned}$$